

DODGE — System Architecture & Product Reference

Data-driven Order-to-cash Discovery & Graph Explorer

Table of Contents

- [1. Product Overview](#)
 - [2. Tech Stack](#)
 - [3. Architecture Overview](#)
 - [4. Data Model](#)
 - [5. Backend Architecture](#)
 - [Ingestion Pipeline](#)
 - [Graph Builder](#)
 - [Query Pipeline](#)
 - [LLM Integration](#)
 - [API Endpoints](#)
 - [6. LLM Prompts — Full Reference](#)
 - [Prompt 1: Intent Extraction](#)
 - [Prompt 2: Cypher Generation \(Custom Queries\)](#)
 - [Prompt 3: Response Synthesis](#)
 - [Prompt 4: Cypher Explanation](#)
 - [7. Cypher Templates](#)
 - [8. Frontend Architecture](#)
 - [9. Safety & Guardrails](#)
 - [10. Configuration](#)
-

1. Product Overview

DODGE is a full-stack application that transforms raw SAP Order-to-Cash (O2C) data into an interactive knowledge graph. Users can:

- **Ingest** JSONL data files → automatically infer schema → build a Neo4j graph
- **Explore** the graph visually — click nodes to inspect properties, expand neighbors
- **Query** the graph in natural language — questions are translated to Cypher via LLM, executed against Neo4j, and results are synthesized back into human-readable answers

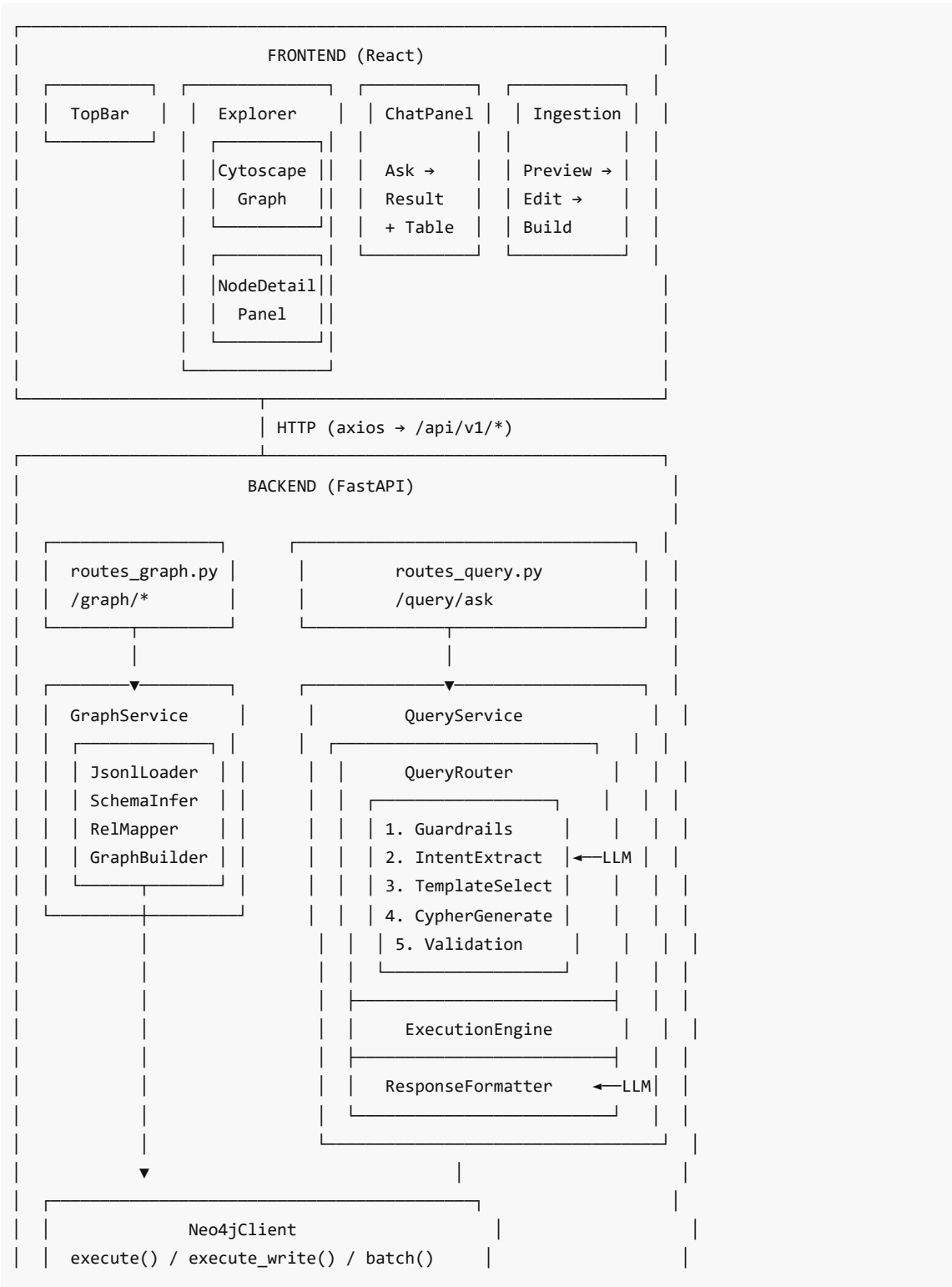
The system models the SAP O2C business process: Customers place Sales Orders, which are Delivered, Invoiced, journaled, and Paid — with Products and Plants as supporting entities.

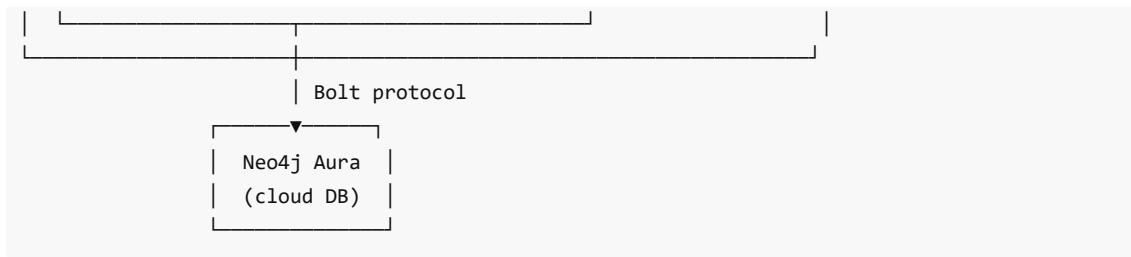
2. Tech Stack

Layer	Technology
Backend	Python 3.12+, FastAPI, Pydantic, Pandas
Database	Neo4j Aura (cloud graph database)
LLM	OpenRouter (primary) + Groq (fallback), OpenAI-compatible API
Frontend	React 19, TypeScript, Vite 8, Tailwind CSS 4

Graph Viz	Cytoscape.js
HTTP	Axios (frontend), Uvicorn (backend)

3. Architecture Overview





4. Data Model

4.1 Node Types (12)

Label	Source Folder	ID Field(s)	Key Properties
Customer	business_partners	businessPartner	businessPartnerName customer, industry, creationDate
Address	business_partner_addresses	[businessPartner, addressId]	cityName, country, postalCode, streetName
SalesOrder	sales_order_headers	salesOrder	soldToParty, totalNetAmount, creationDate, overallDeliveryStatus
SalesOrderItem	sales_order_items	[salesOrder, salesOrderItem]	material, requestedQuantity, netAmount, productCode
Delivery	outbound_delivery_headers	deliveryDocument	actualGoodsMovement, creationDate, overallGoodsMovement
DeliveryItem	outbound_delivery_items	[deliveryDocument, deliveryDocumentItem]	actualDeliveryQuantity, referenceSalesDocument
Invoice	billing_document_headers	billingDocument	totalNetAmount, creationDate, soldToParty, accountingDocument
InvoiceItem	billing_document_items	[billingDocument, billingDocumentItem]	material, netAmount, referenceSalesDocument
JournalEntry	journal_entry_items_accounts_receivable	[companyCode, fiscalYear, accountingDocument, accountingDocumentItem]	amountInTransactionCurrency, postingDate, clearingAccountingDocument
Payment	payments_accounts_receivable	[companyCode, fiscalYear, accountingDocument, accountingDocumentItem]	amountInTransactionCurrency, customer, postingDate

Product	products	product	productType, grossWeight, productGroup, baseUnit
Plant	plants	plant	plantName, salesOrganization, plantCategory

4.2 Relationships (15)

```

Customer --[HAS_ADDRESS]--> Address
Customer --[PLACED_ORDER]--> SalesOrder
SalesOrder --[HAS_ITEM]--> SalesOrderItem
SalesOrderItem --[CONTAINS]--> Product
SalesOrderItem --[PRODUCED_AT]--> Plant
Delivery --[HAS_ITEM]--> DeliveryItem
DeliveryItem --[FULFILLS]--> SalesOrder
DeliveryItem --[SHIPS_FROM]--> Plant
Invoice --[BILLED_TO]--> Customer
Invoice --[HAS_ITEM]--> InvoiceItem
InvoiceItem --[BILLS]--> Delivery
InvoiceItem --[CONTAINS]--> Product
Invoice --[GENERATES]--> JournalEntry
JournalEntry --[CLEARED_BY]--> Payment
Payment --[PAID_BY]--> Customer

```

4.3 Order-to-Cash Flow

```

Customer → SalesOrder → Delivery → Invoice → JournalEntry → Payment → Customer
|           |           |           |           |
|           items       items       items
|           |           |           |
|       SalesOrderItem DeliveryItem InvoiceItem
|           |           |           |
|       Product       Plant       Product
|_____|_____|_____|_____|_____|_____|_____▲

```

5. Backend Architecture

5.1 Ingestion Pipeline

Files: `app/ingestion/jsonl_loader.py` , `app/ingestion/schema_inference.py` , `app/modeling/relationship_mapper.py`

1. **JsonlLoader** — Reads each subdirectory under `data_dir/` as an entity type. Each folder contains `.jsonl` part files, which are parsed line-by-line and concatenated into Pandas DataFrames.
2. **SchemaInference** — Analyzes DataFrames to:
 - Detect ID columns (by name suffix heuristics: `id` , `key` , `document` , `order` , etc.)
 - Find overlapping values between entities (Jaccard similarity on value sets)

- Output ranked `CandidateJoin` list with confidence scores

3. RelationshipMapper — Converts candidates to a `GraphSchema` :

- Builds `NodeSchema` per entity (label, properties, composite ID detection)
- Filters candidate joins (min confidence 0.5, excludes generic fields like `creationDate`)
- Derives `UPPER_SNAKE` edge types from join field names

5.2 Graph Builder

File: `app/graph/graph_builder.py`

- Uses `MERGE` (idempotent) for all node and edge creation
- Supports **composite keys** (`[salesOrder, salesOrderItem]`)
- Creates indexes on ID fields before building
- Processes in batches of 500 records
- `source_entity` field on `NodeSchema` decouples the Neo4j label from the JSONL folder name

5.3 Query Pipeline

File: `app/services/query_service.py` orchestrates the full pipeline:



Intent Types (11)

Intent	Description	Template Used
find_node	Find nodes by label + property filter	find_node.cypher
search	Search all properties of a node type for a value	search.cypher
find_path	Shortest path between two nodes	find_path.cypher
list_neighbors	All connected nodes to a specific node	list_neighbors.cypher
aggregate	SUM/AVG/MIN/MAX on a numeric property	aggregate.cypher
count	Count nodes of a label	count.cypher
find_unlinked	Nodes missing a relationship	find_unlinked.cypher
flow_gaps	O2C incomplete flow analysis	flow_gaps.cypher
rank	Rank nodes by relationship count	rank.cypher
find_latest	Most recent/oldest nodes by date	find_latest.cypher
custom	LLM-generated freeform Cypher	(no template — LLM generates)

Auto-Redirect Logic

The `QueryRouter._correct_template()` method handles misclassifications:

- `aggregate` or `count` with ranking language (most/highest/top) → redirects to `rank`
- `aggregate` with no property + 2+ entities → redirects to `rank`
- `aggregate` with no property + 1 entity → redirects to `count`
- `find_node` with a property not on the entity → redirects to `custom`

Retry Mechanism (Custom Cypher)

When the `custom` path generates invalid Cypher:

1. Generate Cypher via LLM
2. Run schema validation (labels, relationships, variable conflicts)
3. If validation fails, send errors back to LLM with a fix prompt
4. Retry up to 2 times

5.4 LLM Integration

File: `app/llm/llm_client.py`

`FallbackLLMClient` wraps two OpenAI-compatible clients:

Provider	Role	Model	Base URL
OpenRouter	Primary	Configurable via <code>LLM_MODEL</code> env var	<code>https://openrouter.ai/api/v1</code>
Groq	Fallback	<code>llama-3.3-70b-versatile</code>	<code>https://api.groq.com/openai/v1</code>

The client mimics the OpenAI SDK interface (`client.chat.completions.create()`). If the primary call fails for any reason (rate limit, timeout, error), it automatically falls back to Groq.

All LLM calls use `temperature=0.0` and a `timeout` of 15–30 seconds.

5.5 API Endpoints

Base path: `/api/v1`

Graph Endpoints (`/api/v1/graph/`)

Method	Path	Description
POST	<code>/graph/preview</code>	Load data + infer schema (no writes to Neo4j)
POST	<code>/graph/build</code>	Accept schema JSON → build graph in Neo4j
DELETE	<code>/graph/clear</code>	Delete all nodes and relationships
GET	<code>/graph/explore?limit=200</code>	Fetch sample nodes + edges for visualization
GET	<code>/graph/status</code>	Connection status, label counts, node/edge totals
GET	<code>/graph/neighbors/{node_id}?limit=50</code>	Fetch all neighbors of a node by elementId

Query Endpoints (`/api/v1/query/`)

Method	Path	Description
POST	<code>/query/ask</code>	Natural language question → structured response

Request body:

```
{
  "question": "How many sales orders are there?",
  "schema_json": null // optional – defaults to graph_schema_curated.json
}
```

Response:

```
{
  "answer": "There are 250 SalesOrder nodes.",
  "explanation": "Counting all SalesOrder nodes in the graph.",
  "query_used": "MATCH (n:`SalesOrder`) RETURN count(n) AS total",
  "nodes": [],
  "edges": [],
  "data": [{"total": 250}],
  "metadata": {
    "template_name": "count",
    "intent_type": "count",
    "confidence": 0.95,
    "execution_time_ms": 12.5,
    "record_count": 1,
    "node_count": 0,
    "edge_count": 0
  }
}
```

```
}  
}
```

Health Check

Method	Path	Description
GET	/api/v1/health	Returns {status: "ok"}

6. LLM Prompts — Full Reference

The system uses 4 LLM prompts, all defined in `app/llm/prompt_manager.py`. Below is the exact content of each prompt along with its template variables and when it's used.

6.1 Prompt 1: Intent Extraction

Used by: `IntentExtractor.extract()` (called for every user query) **Purpose:** Classify the user's natural language question into a structured intent with entities and filters.

Template variables injected:

Variable	Source	Example Value
{node_labels}	<code>schema.node_labels()</code>	Customer, Address, SalesOrder, SalesOrderItem, Delivery, DeliveryItem, Invoice, InvoiceItem, JournalEntry, Payment, Product, Plant
{edge_types}	<code>schema.edge_types()</code>	HAS_ADDRESS, PLACED_ORDER, HAS_ITEM, CONTAINS, PRODUCED_AT, FULFILLS, SHIPS_FROM, BILLED_TO, BILLS, GENERATES, CLEARED_BY, PAID_BY
{edge_descriptions}	<code>schema.edge_descriptions()</code>	Customer -[HAS_ADDRESS]-> Address (one per line, 15 lines total)
{node_properties}	<code>schema.node_properties_summary()</code>	Customer (id: businessPartner): customer, businessPartnerCategory, ... (one per node type)
{user_query}	User's question	How many orders are linked to Melton Group?

Full prompt text:

You are a query analysis engine for a graph database.

Available node labels: {node_labels}

Available relationship types: {edge_types}

Graph structure (FromNode -[RELATIONSHIP]-> ToNode):
{edge_descriptions}

Node properties (id fields and other properties):
{node_properties}

Given the user query below, extract:

1. intent_type: one of [find_node, search, find_path, list_neighbors, aggregate, count, find_unlinked, flow_gaps, rank, find_latest, custom]
2. entities: list of node labels referenced (must be from the available labels)
3. filters: key-value pairs for filtering. Property names MUST be exact property names from the node properties listed above.
4. confidence: float 0-1 indicating how confident you are

Intent type guide:

- find_node: Find specific nodes by label filtered by a KNOWN property value.
The filter property MUST exist on that node type.
- search: Use when the user provides an ID or value but you are NOT SURE which property it belongs to. Set filters.search_value to the value.
- find_path: Find shortest path between two node types.
- list_neighbors: List nodes connected to a specific node.
- aggregate: Compute SUM/AVG/MIN/MAX on a SPECIFIC numeric property.
Set filters.property_name to the exact property.
- count: Count nodes of a given label.
- find_unlinked: Find nodes MISSING a relationship.
Set filters.relationship_type and filters.target_label.
- flow_gaps: Analyze Order-to-Cash flow for broken/incomplete processes. No filters needed.
- rank: Ranks nodes by relationship count to another type.
ALWAYS include BOTH node types in entities.
Set filters.relationship_type and filters.target_label.
- find_latest: Find most recent or oldest nodes.
Set filters.order_property and filters.order_direction.
- custom: Use for queries that involve TRAVERSING RELATIONSHIPS between node types.
Set filters.description to a brief description.

CRITICAL RULES:

- NEVER use find_node with a property that does NOT exist on that node type.
- Always verify the filter property exists on the target entity before choosing find_node.
- When in doubt between find_node and custom, choose custom.
- Always use exact property names from the node properties list.

Respond ONLY with valid JSON. No explanation.

User query: {user_query}

JSON response:

Expected LLM output (JSON):

```
{
  "intent_type": "custom",
  "entities": ["Customer", "SalesOrder"],
  "filters": {
    "description": "Count orders linked to customer Melton Group"
  },
  "confidence": 0.9
}
```

6.2 Prompt 2: Cypher Generation (Custom Queries)

Used by: `QueryRouter._generate_custom_cypher()` (only for `custom` intent type) **Purpose:** Generate a read-only Cypher query from the user's natural language question.

Template variables injected:

Variable	Source	Example Value
<code>{node_properties}</code>	<code>schema.node_properties_summary()</code>	Customer (id: businessPartner): customer, businessPartnerCategory, ...
<code>{edge_details}</code>	Constructed from <code>schema.edges</code>	Customer -[PLACED_ORDER]-> SalesOrder (join: customer = soldToParty)
<code>{user_query}</code>	User's question	How many orders are linked to Melton Group?

Full prompt text:

```
You are a Neo4j Cypher query expert for a business process graph database.

Graph schema:
Node labels and properties:
{node_properties}

Relationships (FromNode -[TYPE]-> ToNode, joined on source_field=target_field):
{edge_details}

Rules:
- Write a READ-ONLY Cypher query. No CREATE, DELETE, SET, MERGE, DROP.
- Use ONLY the node labels, relationship types, and property names listed above.
- Use backtick-quoted labels and property names: (n:`SalesOrder`), n.`salesOrder`
- Add LIMIT 25 unless the query is an aggregation.
- All property values are stored as strings. Use toString() for comparisons if needed.
- For name matching use CONTAINS (case-sensitive) or toLower() for case-insensitive.
- Return meaningful data – not just node objects. Include relevant properties in RETURN.
- CRITICAL: Use UNIQUE variable names. Never reuse the same variable for a node and a relationship.
  Correct: MATCH (c:`Customer`)-[r:PLACED_ORDER]->(o:`SalesOrder`)
  WRONG:  MATCH (c:`Customer`)-[so:PLACED_ORDER]->(so:`SalesOrder`) ← 'so' used for both!
- Use these variable naming conventions: nodes = lowercase first letter of label (c, o, d, i, p, etc.), relationships = r, r1, r2, etc.

User query: {user_query}

Respond with ONLY the Cypher query. No explanation, no markdown fences.
```

Expected LLM output:

```
MATCH (c:`Customer`)-[r:PLACED_ORDER]->(o:`SalesOrder`)
WHERE c.`businessPartnerName` CONTAINS 'Melton Group'
```

```
RETURN COUNT(o) AS orderCount
```

6.3 Prompt 3: Response Synthesis

Used by: `ResponseFormatter._llm_answer()` (after query execution) **Purpose:** Convert raw Cypher results into a natural language answer.

Template variables injected:

Variable	Source	Example Value
{user_query}	Original user question	How many sales orders are there?
{cypher}	The Cypher that was executed	<code>MATCH (n:\SalesOrder`) RETURN count(n) AS total`</code>
{results_json}	JSON of first 10 result rows	<code>[{"total": 250}]</code>
{total_count}	Total number of result rows	1

Full prompt text:

You are a data analyst assistant for a business process graph database (SAP Order-to-Cash: Customers, Sales Orders, Deliveries, Invoices, Journal Entries, Payments, Products, Plants).

The user asked: {user_query}

The following Cypher query was executed:
{cypher}

Here are the results (up to 10 rows):
{results_json}

Total result count: {total_count}

Using ONLY the data above, write a concise natural language answer to the user's question. Be specific – include actual values, names, and numbers from the results. If the results are empty, say so clearly. Do NOT make up data that is not in the results. Keep it under 3 sentences unless a list is needed.

Expected LLM output:

There are 250 sales orders in the system.

6.4 Prompt 4: Cypher Explanation

Used by: Not currently wired into the pipeline but available for future use. **Purpose:** Generate a one-sentence explanation of what a Cypher query does.

Template variables: {cypher}

Full prompt text:

Given this Cypher query:

```
{cypher}
```

Explain in one plain-English sentence what this query does.

7. Cypher Templates

All templates are stored in `backend/templates/cypher/` as `.cypher` files with frontmatter headers.

find_node.cypher

```
-- name: find_node
-- description: Find nodes by label and optional property filter
-- params: label, property_name, property_value
MATCH (n:`${label}`)
WHERE n.`${property_name}` = '${property_value}'
RETURN n
LIMIT 25
```

search.cypher

```
-- name: search
-- description: Search for a node by value across all its properties
-- params: label, search_value
MATCH (n:`${label}`)
WHERE ANY(key IN keys(n) WHERE toString(n[key]) = '${search_value}')
RETURN n
LIMIT 25
```

count.cypher

```
-- name: count
-- description: Count nodes of a given label
-- params: label
MATCH (n:`${label}`)
RETURN count(n) AS total
```

aggregate.cypher

```
-- name: aggregate
-- description: Aggregate a numeric property on nodes of a given label
-- params: label, aggregate_function, property_name
MATCH (n:`${label}`)
WHERE n.`${property_name}` IS NOT NULL
RETURN ${aggregate_function}(toFloat(n.`${property_name}`)) AS result, count(n) AS
total_nodes
```

find_path.cypher

```
-- name: find_path
-- description: Find shortest path between two node types
-- params: start_label, end_label
MATCH (a:`${start_label}` `{${start_property}`: '${start_value}'}),
      (b:`${end_label}` `{${end_property}`: '${end_value}'}),
      path = shortestPath((a)-[*..10]-(b))
RETURN path
LIMIT 5
```

list_neighbors.cypher

```
-- name: list_neighbors
-- description: List all nodes directly connected to a given node
-- params: label, property_name, property_value
MATCH (n:`${label}` `{${property_name}`: '${property_value}'})-[r]-(m)
RETURN n, r, m
LIMIT 50
```

rank.cypher

```
-- name: rank
-- description: Rank nodes by number of relationships to another node type
-- params: label, relationship_type, target_label
MATCH (n:`${label}`)-[r:`${relationship_type}`]-(m:`${target_label}`)
RETURN n, count(r) AS relationship_count
ORDER BY relationship_count DESC
LIMIT 25
```

find_unlinked.cypher

```
-- name: find_unlinked
-- description: Find nodes that lack a specific relationship
-- params: label, relationship_type, target_label
MATCH (n:`${label}`)
WHERE NOT EXISTS { MATCH (n)-[:`${relationship_type}`]-(:`${target_label}`) }
RETURN n
LIMIT 25
```

flow_gaps.cypher

```
-- name: flow_gaps
-- description: Find sales orders with incomplete O2C flows
-- params: (none)
MATCH (so:SalesOrder)
```

```
OPTIONAL MATCH (so)<-[:FULFILLS]-(di:DeliveryItem)<-[:HAS_ITEM]-(d:Delivery)
OPTIONAL MATCH (d)<-[:BILLS]-(ii:InvoiceItem)<-[:HAS_ITEM]-(inv:Invoice)
OPTIONAL MATCH (inv)-[:GENERATES]->(je:JournalEntry)
OPTIONAL MATCH (je)-[:CLEARED_BY]->(pay:Payment)
WITH so,
    count(DISTINCT d) > 0 AS has_delivery,
    count(DISTINCT inv) > 0 AS has_invoice,
    count(DISTINCT je) > 0 AS has_journal_entry,
    count(DISTINCT pay) > 0 AS has_payment
WHERE NOT (has_delivery AND has_invoice AND has_journal_entry AND has_payment)
RETURN so.salesOrder AS sales_order,
    has_delivery, has_invoice, has_journal_entry, has_payment,
    CASE
        WHEN has_delivery AND NOT has_invoice THEN 'Delivered but not billed'
        WHEN has_invoice AND NOT has_delivery THEN 'Billed without delivery'
        WHEN NOT has_delivery THEN 'No delivery'
        WHEN has_invoice AND NOT has_journal_entry THEN 'Billed but no journal entry'
        WHEN has_journal_entry AND NOT has_payment THEN 'Journal entry but no payment'
        ELSE 'Incomplete flow'
    END AS gap_type
ORDER BY gap_type
LIMIT 50
```

find_latest.cypher

```
-- name: find_latest
-- description: Find most recent nodes ordered by a date property
-- params: label, order_property, order_direction
MATCH (n:`${label}`)
WHERE n.`${order_property}` IS NOT NULL
RETURN n
ORDER BY n.`${order_property}` ${order_direction}
LIMIT 10
```

8. Frontend Architecture

8.1 Pages

Page	Route	Purpose
Explorer	/	Main page — graph visualization + chat panel + node detail
Ingestion	/ingest	Two-step data loading: preview schema → edit JSON → build graph

8.2 Components

Component	Description
TopBar	Navigation bar with DODGE branding and page links

CytoscapeGraph	Cytoscape.js wrapper with COSE layout, node coloring by label, click handlers, imperative <code>addElements()</code> for neighbor expansion
ChatPanel	Chat interface — sends questions to <code>/query/ask</code> , displays answer + explanation + Cypher + data table
NodeDetailPanel	Sidebar showing selected node's properties + "Expand Neighbors" button

8.3 Explorer Page State

- **fullNodes/fullEdges** — The complete graph from `/graph/explore` (loaded on mount)
- **queryNodes/queryEdges** — Subgraph from the last chat query result
- **viewMode** — Toggle between "Full Graph" and "Query Results"
- **selectedNode** — The node clicked on in the graph (shows NodeDetailPanel)
- **expandedNodeIds** — Set of nodes whose neighbors have been fetched

8.4 Graph Visualization Details

- Nodes are colored by label using a 10-color palette
- Node display names are derived from business properties (e.g., `businessPartnerName`, `salesOrder`, `plantName`)
- Selected node gets a yellow border + size increase
- Neighbor expansion uses `addElements()` (imperative) with concentric layout animation
- COSE layout for initial render with `nodeRepulsion: 8000`

9. Safety & Guardrails

9.1 Input Guardrails

- Max query length: 1000 characters
- Blocked patterns: "ignore previous", "forget your instructions", "system prompt" (prompt injection defense)

9.2 Domain Guardrails

- All entities extracted by the LLM must match known node labels in the schema

9.3 Output Guardrails (Cypher)

- **Blocked keywords:** `DELETE`, `DETACH DELETE`, `DROP`, `CREATE INDEX`, `CREATE CONSTRAINT`, `CALL dbms`, `LOAD CSV`
- **No semicolons** (prevents multi-statement injection)
- **Must start with a read operation:** `MATCH`, `OPTIONAL MATCH`, `WITH`, `UNWIND`, `RETURN`, or `CALL {`

9.4 Schema Validation

- All node labels in the Cypher must exist in the schema
- All relationship types must exist in the schema
- **Variable conflict detection:** Same variable name cannot be used for both a node and a relationship (catches common LLM error)

9.5 Cypher Template Safety

- Templates use `${param}` placeholders filled by `CypherGenerator`
- Single quotes in values are escaped to prevent injection

- Empty parameter cleanup via regex (removes invalid `{` `: '...'}` patterns)

10. Configuration

Environment Variables (`.env`)

```
# Neo4j Aura
NEO4J_URI=neo4j+s://xxxxxxx.databases.neo4j.io
NEO4J_USER=neo4j
NEO4J_PASSWORD=your-password

# LLM - Primary (OpenRouter)
OPENROUTER_API_KEY=sk-or-v1-...
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
LLM_MODEL=nvidia/nemotron-3-nano-30b-a3b:free

# LLM - Fallback (Groq)
GROQ_API_KEY=gsk_...
GROQ_BASE_URL=https://api.groq.com/openai/v1
GROQ_MODEL=llama-3.3-70b-versatile

# Data
DATA_DIR=./sap-order-to-cash-dataset/sap-o2c-data
CYPHER_TEMPLATE_DIR=templates/cypher

# General
LOG_LEVEL=INFO
```

Running the Application

Backend:

```
cd backend
uvicorn app.main:app --reload
```

Frontend:

```
cd frontend
npm install
npm run dev
```

The Vite dev server proxies `/api/v1/*` requests to the backend.

File Structure

```
dodge/
├── backend/
│   └── app/
```



```

| | | └─ api/
| | | | └─ routes_graph.py          # Graph management endpoints
| | | |   └─ routes_query.py        # NL query endpoint
| | | └─ config/
| | | | └─ constants.py             # App constants, blocked keywords
| | | |   └─ settings.py            # Pydantic settings from .env
| | | └─ graph/
| | | | └─ graph_builder.py         # MERGE nodes/edges into Neo4j
| | | |   └─ neo4j_client.py        # Neo4j driver wrapper
| | | └─ ingestion/
| | | | └─ jsonl_loader.py          # Read JSONL → DataFrames
| | | |   └─ schema_inference.py    # ID detection + join inference
| | | └─ llm/
| | | | └─ cypher_generator.py      # Fill template placeholders
| | | | └─ guardrails.py            # Input/domain/output validation
| | | | └─ intent_extractor.py      # LLM intent classification
| | | | └─ llm_client.py            # FallbackLLMClient (OpenRouter + Groq)
| | | | └─ prompt_manager.py        # 4 built-in prompt templates
| | | |   └─ template_registry.py   # Load + select .cypher templates
| | | └─ modeling/
| | | | └─ graph_schema.py          # NodeSchema, EdgeSchema, GraphSchema
| | | |   └─ relationship_mapper.py # Build schema from inferred joins
| | | └─ query/
| | | | └─ execution_engine.py      # Execute Cypher + time it
| | | | └─ path_extractor.py        # Extract nodes/edges from results
| | | | └─ query_router.py          # Full routing pipeline + retry
| | | | └─ query_validator.py       # Schema + variable validation
| | | |   └─ response_formatter.py  # LLM answer + fallback formatting
| | | └─ services/
| | | | └─ graph_service.py         # Orchestrate full ingestion
| | | |   └─ query_service.py       # Orchestrate query pipeline
| | | └─ utils/
| | | | └─ helpers.py              # sanitize_label, ensure_directory
| | | |   └─ logger.py             # Logging setup
| | | └─ main.py                  # FastAPI app entry point
| └─ templates/cypher/            # 10 Cypher query templates
| └─ graph_schema_curated.json    # Curated schema (12 nodes, 15 edges)
|   └─ graph_schema.json          # Auto-inferred schema (backup)
└─ frontend/
  └─ src/
    └─ api/client.ts              # Axios API client (7 functions)
    └─ components/
      └─ ChatPanel.tsx            # Chat UI + DataTable
      └─ CytoscapeGraph.tsx      # Cytoscape wrapper + expansion
      └─ NodeDetailPanel.tsx     # Node property inspector
      └─ TopBar.tsx              # Navigation bar
    └─ pages/
      └─ Explorer.tsx             # Main page (graph + chat + detail)
      └─ Ingestion.tsx           # Data loading wizard
    └─ types/index.ts            # TypeScript interfaces
    └─ App.tsx                   # Router setup

```

```
| | └─ main.tsx          # React entry point
| └─ package.json
|
└─ graph_schema_curated.json  # (duplicate at root for schema loading)
└─ SYSTEM.md                 # ← This document
```